

Project 5 (C++): You are to implement Dijkstra's Single-source-shortest paths algorithm.

Problem Statement: Given a directed graph, $G = \langle N, E \rangle$, and the source node, S , in G , the task is to find the shortest paths from S to all other nodes in G .

*** You will be given 2 data files: data1 and data2. The graph in data1 has 5 nodes and graph in data2 has 8 nodes.

What you need to do:

** You will be given 2 data files: SSS_data1 and SSS_data2.

Illustrations:

1) Tree search illustration:

*** Use a blank paper to draw an illustration as follows:

- on the upper-left of the paper, draw the directed graph in data1. On the upper-right of paper, draw the cost-matrix of the graph in data1.
- Below the graph, draw the search tree use source node, 2, as the root to find the shortest paths from node 4 to all other nodes in G . Then, at the bottom of the page, list the paths: 2 to 1, 2 to 2, 2 to 3, 2 to 4 and 2 to 5 with costs.

2) Array implementation of the tree search illustration:

*** Use another page of blank paper to draw another illustration as follows:

- on the upper-left of the paper, draw the directed graph in data1. On the upper-right of paper, draw the cost-matrix of the graph in data1.
- Below the graph draw 3 1-D arrays (ToDo, Best, parent), use source node 2, follow the algorithm steps to find the shortest paths, as taught in class. at the bottom of the page, list the paths: 2 to 1, 2 to 2, 2 to 3, 2 to 4 and 2 to 5 with costs.
- Compare the shortest paths in 2) with the shortest paths from the tree-search in 1). Both should be identical.

5) Run your program with data1, enter the source node from console in the sequence: 1, 3, 5, 2, 4, 0.

6) Run your program with data2, enter the source node from console in the sequence: 1, 3, 5, 7, 2, 4, 6, 8, 0.

*** Include in your hard copy:

- cover page (Include only main () algorithm steps)
- Illustration from 1) (-1 if omitted.)
- Illustration from 2) (-1 if omitted.)
- source code
- print data1 // with caption: ** Below is data1
- SSSfile for data1 // with caption: ** Below is SSS file for data1
- debugFile for data1 // with caption: ** Below is debugFile for data1. Limited to 3 pages if more.
- print data2 // with caption: ** Below is data2
- SSSfile for data2 // with caption: ** Below is SSS file for data2
- debugFile for data2 // with caption: ** Below is debugFile for data2. Limited to 3 pages if more.

Language: C++

Project Name: Dijkstra's Single-source-shortest paths

Project points: 10 pts

Due Date: Soft copy (*.zip) and hard copy (word file) :

- 0 (10/10 pts): on time, 3/21/2026 Saturday before midnight, 11:59pm.
- (-10/10 pts): non-submission, 3/21/2026 Saturday after midnight

*** All submission MUST include Soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as stated in the project submission requirement; otherwise, your submission will be rejected.

Email subject: (323) firstName lastName <Project 5: Dijkstra's Single-source-shortest paths (C++)>

*** Inside the email body includes:

- Your answer to the five questions (-2 if omitted)
- Screen recording link. (-2 without the recording!)

*** Place your screen recording link in your project submission email body below the 5 questions.

I. Inputs: a) inFile (argv [1]): a directed graph, represented by a list of edges with costs, $\{<n_i, n_j, c>\}$
// You may assume that nodes' Id is from 1 to N (0 is not used)

The format of the input file is as follows:

The first text line is the number of nodes, N, follows by a list of triplets, $<n_i, n_j, cost>$

For example:

```
5 // there are 5 nodes in the graph
1 5 10 // an edge from node 1 to node 5, the cost is 10
2 3 5
1 2 20
3 5 2
:
:
```

b) source: **Given by the user from Console.**

II. Outputs:

a) SSSfile (argv [2]) : The single-source-shortest shortest paths from a given source node.

For example, if there are 6 nodes in the graph G, and the given source node is 1,
then your SSSfile will have the followings:

```
=====
There are 6 nodes in the input graph.
=====
Source node = 1
The shortest paths from the source node 1 are:
The path from 1 to 1 : 1 ← 1 : cost = 0
The path from 1 to 2 : 2 ← ... ← 1: cost = whatever
The path from 1 to 3 : 3 ← ... ← 1: cost = whatever
:
:
The path from 1 to 6 : 6 ← ... ← 1: cost = whatever
```

Or, if there are 8 nodes in the graph G, and the given source node is 5,
then your SSSfile will have the followings:

```
=====
There are 8 nodes in the input graph.
=====
Source node = 5
The shortest paths from node 5 are:
The path from 5 to 1 : 1 ← ... ← 5: cost = whatever
The path from 5 to 2 : 2 ← ... ← 5: cost = whatever
The path from 5 to 3 : 3 ← ... ← 5: cost = whatever
:
:
The path from 5 to 8 : 8 ← ... ← 5 : cost = whatever
```

b) deBugFile (argv [3]): As specs dictates.

III. Data structure:

1) A DijkstraSSS class

- (int) numNodes //number of nodes in G
- (int) sourceNode
- (int) minNode
- (int) currentNode
- (int) newCost
- (int **) costMatrix
 - // a 2-D cost matrix (integer array), size of N+1 X N+1, should be dynamically allocated.
 - // Initially, costMatrix[i][i] set to zero and all others set to infinity, 999
 - // Note: 0 is not used for node Id.
- (int*) father // a 1-D integer array, size of N+1, should be dynamically allocated.
- (int*) toDo // 1-D integer array, size of N+1, should be dynamically allocated;
 - //initially set to 1 (shortest paths not found yet.)
- (int*) best // a 1-D integer array, size of N+1, should be dynamically allocated.
 - // to store the cheapest cost so far from source to each other nodes. It may change during the search.

Methods:

- constructor (...) // use as you need.
- loadCostMatrix (. . .)// read from input file and fill the costMatrix,
- setBest (sourceNode) // copy the row of sourceNode from costMatrix.
- setFather (sourceNode) // set all to sourceNode.
- setToDo (sourceNode) // set toDo [sourceNode] to 0 and all other to 1.
- int findMinNode (. . .) // find a node with minimum cost in best[i] where toDo[i] == 1
 - // Algorithm is given below
- (bool) checkToDo (...) // the method returns true if ToDo[i] are all == 0, otherwise returns false. **On your own.**
- debugPrint (...) // Prints father array to deBugFile in the following format:
 - *** Below is father Array ***

node	father
1	1
2	3
3	1
:	:

 - // Prints ToDo array (with proper heading). Use the same format as above.
 - // Prints Best array (with proper heading). Use the same format as above.
- printShortestPath (...) // on your own.
 - // The method traces each node in G back to sourceNode via father.
 - // print to SSSfile, the shortest path from each to sourceNode with the total cost.
 - //using the format given in the above. You should know how to do this method.

V. main (...)

```

step 0: inFile, SSSfile, deBugFile ← open via argv []
        numNodes ← get from inFile
        Allocate and initialize all members in the DijkstraSSS class accordingly
        loadCostMatrix (inFile)
step 1: console ← “Please enter the source node (integer)” //Ask user for a source node. when done enter 0.
        sourceNode ← from console
        SSSfile ← “In main (): Source Node is” // write sourceNode
        if sourceNode <= 0
            close all files and exit the program
step 2: setBest (sourceNode)
        setFather (sourceNode)
        setToDo(sourceNode)
        deBugPrint (...)
step 3: minNode ← findMinNode (... , deBugFile)
        SSSfile ← “In main (): minNode found is” // write minNode
        toDo[minNode] ← 0
step 4: // expanding the minNode
        currentNode ← 1
        SSSfile ← “In main (): currentNode is” // write currentNode
step 5: if toDo[currentNode] == 1 {
        newCost ← best [minNode] + costMatrix [minNode, currentNode]
        if newCost < best [currentNode]
            best[currentNode] ← newCost
            father[currentNode] ← minNode
            debugPrint (...)
        }
step 6: currentNode ++
step 7: repeat step 5 to step 6 while currentNode <= numNodes
step 8: repeat step 3 to step 7 until checkToDo (...) == true
        // begin printing the paths from sourceNode
step 9: printShortestPath (sourceNode, SSSfile)
step 10: repeat step 1 to step 9 while sourceNode > 0
step 11: close all files

```

V. (int) findMinNode (... , deBugFile)

```

Step 0: deBugFile ← “entering findMinNode ()”
        minCost ← 999
        minNode ← 0
Step 1: index ← 1
Step 2: if toDo[index] == 1 // not done yet
        if best[index] < minCost
            minCost ← best[index]
            minNode ← index
Step 3: index++
Step 4: repeat step 2 to step 3 while index <= numNodes
Step 5: deBugFile ← “In findMinNode () minNode found is” //write minNode
Step 6: return minNode

```